

Bluff-audit — native-build fallback cryptic C-link death (§11.4.138)

Revision: 1 **Last modified:** 2026-06-29T11:00:00Z **Scope:**

scripts/setup.sh native-build fallback · scripts/install_deps.sh

Trigger: operator-escape — the GREEN suite passed while the operator hit a real, cryptic, unactionable failure (§11.4.138 → §11.4.102 → §11.4.115 + permanent guard).

1. The operator-found defect (FACT)

Running `bash scripts/setup.sh` on a base **ALT Linux** host where:

1. rootless Podman had exhausted `/etc/subuid + /etc/subgid`, so the containerized build failed (`lchown ... invalid argument / newuidmap: executable file not found`); then
2. the §11.4.101 **native fallback** (`build_native.sh → ./configure`) died with:

```
checking for gcc... gcc
checking whether the C compiler works... no
configure: error: C compiler cannot create executables
```

2. Root cause (systematic-debugging, §11.4.102 — proven, not guessed)

`gcc` was present but **could not LINK an executable** because the host lacked the C-runtime dev objects. **Verified as FACT on this identical ALT host:**

```
$ rpm -q --whatprovides /usr/lib64/crt1.o
glibc-devel-2.40.0.224.573a-alt1.x86_64
```

So `glibc-devel` (which owns `crt1.o`) was absent → the linker cannot produce an executable → `autoconf's AC_PROG_CC` link check fails → the cryptic abort.

setup.sh's native path invoked ./configure (via build_native.sh) **with no preflight that verifies the compiler can link**, so the operator received a bare autoconf error naming **neither the cause (missing libc dev objects) nor the fix (install glibc-devel)** — a \$11.4.6 honesty gap + robustness defect.

Pre-fix call sites (quoted from git show 9900fd1:scripts/setup.sh, Step 2 Linux):

```
if ! bash scripts/build_containerized.sh; then
    echo "[setup] ▲ containerized build failed ..."
    ...
    bash scripts/build_native.sh          # ← runs ./configure with
        NO link preflight
fi
...
else
    bash scripts/build_native.sh          # ← no-engine path, same
        gap
fi
```

build_native.sh:243 then runs ./configure --prefix="\$BUILD_DIR" --disable-debug — the autoconf cc-works check that dies cryptically.

Compounding finding: the existing install_deps.sh ALT package list (9900fd1:scripts/install_deps.sh:60) was ITSELF broken for this host — it **omitted glibc-devel** (the crt1.o owner) and listed jemalloc-devel + byacc, **neither of which exists on ALT** (apt-cache show MISSING), so the whole apt-get install would have failed. So even the documented remedy (install_deps.sh) would not have fixed the operator's host.

3. The bluff-audit — which assertion should have caught it, and didn't

Verdict: NO existing test fault-injected a non-linking C compiler or exercised the native-build fallback's toolchain assumptions. The GREEN suite was blind to this entire class. The two closest tests, cited to file:line:

Test	Why it did NOT catch it
scripts/tests/ 42_setup_install_uninstall_e2e.sh:15-25	An explicit RECURSION GUARD — "This test does NOT invoke bash scripts/setup.sh"; it mirrors the install-artifact generators on a healthy host, never

Test	Why it did NOT catch it
scripts/tests/67_local_deps.sh:52-55,82	reaches Step 2 native build, never fault-injects a broken toolchain. Uses a C compiler for the C3 probe but SKIPS-with-reason when the compiler is absent and assumes a working cc ; it validates the dependency- <i>obtain</i> mechanism, not the native-build link preflight , and never asserts an honest message for a compile-but-can't-link host.

A repo-wide grep confirms the gap (only run_all.sh's error string mentions build_native): no test referenced cannot create executables, cc_can_link, build-essential, glibc-devel, or a non-linking compiler before this fix.

The native fallback itself shipped (commit 5363d0b) with the honest boundary "native-fallback validated as parse-clean + logically correct; full native GREEN requires a host with libevent-dev + libncurses-dev" — i.e. it was **knowingly merged WITHOUT runtime coverage of the can't-link case**. That gap is the bluff.

4. The fix + permanent regression guard (§11.4.115 / §11.4.135)

- **scripts/setup.sh** — added cc_can_link() + _native_build_preflight(): compiles+links a trivial int main() {return 0;} BEFORE build_native.sh. On failure it emits an **honest, per-distro actionable message** (names glibc-devel/ libc6-devel; macOS xcode-select --install per §11.4.81) and, consent-gated (interactive prompt / TMX_AUTO_INSTALL_DEPS=1 / --install-deps), **auto-installs** via install_deps.sh, then re-checks. Exits 5 (clear), not a cryptic autoconf death. Guards all three native-build sites (Darwin + Linux fallback + Linux no-engine).
- **scripts/install_deps.sh** — corrected ALT package set (rpm-verified: gcc glibc-devel make libevent-devel libncursesw-devel autoconf automake pkg-config bison flex; shadow-submap for rootless newuidmap), per-distro mapping (ALT/Debian/Fedora/

Arch/openSUSE/Alpine/macOS), idempotent skip-present, install-only (§11.4.122 — zero remove verbs), honest-on-failure, INSTALL_DEPS_DRY_RUN.

- **scripts/tests/70_native_fallback_cc_link.sh** — the permanent guard (§11.4.135). RED_MODE=1 reproduces the defect on a preflight-neutered artifact; RED_MODE=0 is the standing GREEN guard asserting the honest refusal is present on current code; 13/13 PASS ×3 deterministic; self-contained §1.1 paired mutation (neuter cc_can_link → honest refusal vanishes → guard FAILs = MUTATION CAUGHT).

5. Evidence

Under qa-results/loop-20260629/native-fallback-fix/
RED_autoconf_cc_cannot_create_executables.log (verbatim cryptic death), RED_prefix_HEAD_no_honest_refusal.txt (pre-fix 9900fd1 emits no honest refusal — the genuine broken artifact),
GREEN_preflight_fakecc.log, GREEN_autoinstall_wiring.log,
GREEN_autoinstall_success.log, fake_nonlinking_cc.sh (host-safe fault injector).

Sources verified 2026-06-29

- GNU Autoconf 2.71 AC_PROG_CC / _AC_COMPILER_EXEEXT (the cc-works check that emits "C compiler cannot create executables").
- ALT package facts verified locally via apt-cache show + rpm -q --whatprovides on the identical ALT 11 host (not guessed — §11.4.6).